# DATABRICKS PYSPARK CHEAT SHEET

Production Reference: SparkSession, DataFrame API, Schemas, Joins, Windowing & Aggregations

PYSPARK | DATAFRAME API | DELTA LAKE 3.X | AUTO LOADER

## SparkSession & Standard Imports

Setup

```python
from pyspark.sql import SparkSession, Window as W
from pyspark.sql import functions as F, types as T
from delta.tables import DeltaTable

# Pre-initialized in Databricks as 'spark'; Configured for Medallion
spark.conf.set("spark.sql.shuffle.partitions", "auto")
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")
spark.conf.set("spark.databricks.delta.autoCompact.enabled", "true")
```

- `optimizeWrite` performs an exchange before writing to ensure 128MB-1GB files.
- `autoCompact` automatically checks and compacts files after individual write stages.

## DataFrame Projections & Filtering

Core API

```python
# Projections, Expressions & Casts
df_clean = (df
    .select(
        "customer_id",
        F.col("amount").cast(T.DecimalType(18, 2)).alias("amount_eur"),
        F.to_date(F.col("created_at")).alias("txn_date")
    )
    .withColumn("tier",
        F.when(F.col("amount_eur") > 5000, "PLATINUM")
         .when(F.col("amount_eur") > 1000, "GOLD")
         .otherwise("STANDARD")
    )
    .filter((F.col("amount_eur") > 0) & F.col("customer_id").isNotNull())
    .dropDuplicates(["customer_id", "txn_date"])
    .orderBy(F.col("amount_eur").desc_nulls_last())
)
```

**withColumns():** In Spark 3.3+, pass a dictionary to `df.withColumns(...)` to add multiple columns in a single projection pass!

## Schema Definition (StructType & StructField)

Data Quality

```python
# Explicit schema avoids expensive inferSchema full-scans
txn_schema = T.StructType([
    T.StructField("txn_id", T.LongType(), nullable=False),
    T.StructField("cust_id", T.StringType(), nullable=False),
    T.StructField("amount", T.DecimalType(18, 2), nullable=False),
    T.StructField("attributes", T.MapType(T.StringType(), T.StringType())),
    T.StructField("items", T.ArrayType(T.StringType())),
    T.StructField("event_time", T.TimestampType())
])

# Read with enforced schema and corrupt record safety
df = (spark.read
      .schema(txn_schema)
      .option("mode", "PERMISSIVE")
      .option("columnNameOfCorruptRecord", "_corrupt")
      .json("abfss://landing@adls.dfs.core.windows.net/txns/"))
```

**FinOps Sizing:** Enforcing schemas on read eliminates a 2-pass scan over multi-terabyte JSON/CSV landing zones.

## pyspark.sql.functions (F) Master Reference

Functions

| Category | Top Functions |
|---|---|
| String | `F.concat_ws('_', c1, c2)`, `F.regexp_replace()`, `F.split()`, `F.trim()` |
| Date/Time | `F.to_date()`, `F.date_add(c, 30)`, `F.datediff(c1, c2)`, `F.date_trunc('MM')` |
| Nulls | `F.coalesce(c1, c2, F.lit('N/A'))`, `df.na.fill(0)`, `df.na.drop()` |
| Array | `F.explode()`, `F.array_contains()`, `F.size()`, `F.array_distinct()` |
| JSON | `F.from_json(col, schema)`, `F.to_json(struct)`, `F.get_json_object()` |

**Vectorization:** Built-in `pyspark.sql.functions` execute natively in JVM/C++ Photon without Python serialization penalties!

## Joins, Broadcast Hints & Skew Salting

Optimization

```python
# Broadcast Join: Eliminates shuffle for lookup tables (<100MB)
df_joined = df_fact.join(F.broadcast(df_dim), on="cat_id", how="left")

# Semi & Anti Joins (Replaces expensive IN / NOT IN subqueries)
df_active = df_users.join(df_purchases, on="user_id", how="left_semi")
df_churned = df_users.join(df_purchases, on="user_id", how="left_anti")

# Manual 2-Stage Skew Salting (When AQE cannot balance 100M+ key)
df_skewed = df_fact.withColumn("salt", (F.rand() * 10).cast("int"))
df_dim_exploded = (df_dim
    .withColumn("salt_arr", F.array([F.lit(i) for i in range(10)]))
    .withColumn("salt", F.explode("salt_arr")))
df_salted_join = df_skewed.join(df_dim_exploded, on=["key", "salt"])
```

## Analytical Window Specifications (pyspark.sql.Window)

Analytics

```python
# Define Window Specifications
w_dept = W.partitionBy("dept_id").orderBy(F.col("salary").desc())
w_roll_7d = (W.partitionBy("account_id")
              .orderBy(F.col("txn_date").cast("timestamp").cast("long"))
              .rangeBetween(-7 * 86400, 0)) # 7 days in seconds

df_analyzed = (df
    .withColumn("dense_rank", F.dense_rank().over(w_dept))
    .withColumn("prev_salary", F.lag("salary", 1, 0).over(w_dept))
    .withColumn("salary_diff", F.col("salary") - F.col("prev_salary"))
    .withColumn("rolling_7d_total", F.sum("amount").over(w_roll_7d))
)
```

**RangeBetween vs RowsBetween:** `rangeBetween` handles calendar days and gaps; `rowsBetween` operates strictly on physical row indices.

# DATABRICKS PYSPARK CHEAT SHEET

DeltaTable API, Auto Loader Streaming, Vectorized UDFs & Performance Tuning

LAKEHOUSE API | PYARROW VECTORIZED | STREAMING

## DeltaTable Python API & Upserts

Lakehouse

```python
from delta.tables import DeltaTable

dt = DeltaTable.forName(spark, "prod_catalog.silver.customers")

# Pythonic ACID Merge / Upsert
(dt.alias("t")
   .merge(source_df.alias("s"), "t.cust_id = s.cust_id")
   .whenMatchedUpdate(
       condition="s.op = 'U'",
       set={"name": "s.name", "updated_at": "current_timestamp()"}
   )
   .whenMatchedDelete(condition="s.op = 'D'")
   .whenNotMatchedInsert(
       values={"cust_id": "s.cust_id", "name": "s.name",
               "created_at": "current_timestamp()"}
   )
   .execute())

# Maintenance & Time Travel
dt.optimize().executeCompaction()
dt.vacuum(168)  # 7 days
hist_df = dt.history()
```

## Auto Loader & Structured Streaming

Streaming

```python
# Ingestion with Databricks Auto Loader (cloudFiles)
stream_df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/checkpoints/schema")
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load("abfss://raw@adls.dfs.core.windows.net/events/"))

# Watermarked Sink write with AvailableNow trigger
query = (stream_df
    .withWatermark("event_time", "10 minutes")
    .writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/checkpoints/bronze_events")
    .trigger(availableNow=True) # Incremental batch
    .toTable("bronze.events"))
```

**Trigger(availableNow=True):** Replaces legacy `once=True`. Processes all pending data in rate-limited micro-batches then stops, slashing cluster costs!

## PyArrow Vectorized Pandas UDFs

Performance

```python
import pandas as pd
from pyspark.sql.functions import pandas_udf

# Vectorized Series-to-Series UDF (Uses Apache Arrow)
@pandas_udf(T.DoubleType())
def calculate_vat(price: pd.Series, rate: pd.Series) -> pd.Series:
    return price * (1.0 + rate)

df_taxed = df.withColumn("gross", calculate_vat(F.col("net"), F.col("rate")))

# Grouped Map Vectorized UDF (applyInPandas)
def normalize_group(pdf: pd.DataFrame) -> pd.DataFrame:
    pdf["z_score"] = (pdf["val"] - pdf["val"].mean()) / pdf["val"].std()
    return pdf

df_norm = df.groupby("device_id").applyInPandas(
    normalize_group, schema=output_schema
)
```

**Avoid Standard @udf:** Regular Python UDFs force row-by-row Py4J socket serialization. Pandas UDFs process columnar Arrow batches up to 100x faster!

## Partition Management: Repartition vs Coalesce

FinOps / Sizing

```python
# Inspect current partitions
num_parts = df.rdd.getNumPartitions()

# Repartition: FULL SHUFFLE across cluster (Increase/Balance)
df_balanced = df.repartition(200)
df_hash = df.repartition(100, "region")

# Coalesce: ZERO SHUFFLE (Shrinks local partition blocks)
df_shrunk = df.coalesce(10) # Use right before file write!
```

| Operation | Network Shuffle | Partition Target |
|---|---|---|
| `repartition(N)` | Yes (Exchange) | Increase or redistribute evenly |
| `coalesce(N)` | No (Local merge) | Shrink only; avoids small files |

**Rule of Thumb:** Target **100MB - 200MB** uncompressed data per partition.

## Caching, Persistence Levels & Cleanup

Memory Management

```python
from pyspark import StorageLevel

# Cache: MEMORY_AND_DISK_DESER (Default)
df_cached = df.cache()

# Persist with Serialized Kryo (Saves up to 3x RAM)
df_persisted = df.persist(StorageLevel.MEMORY_ONLY_SER)

# MANDATORY: Evict from RAM when actions complete!
df_cached.unpersist()
```

- **When to Cache:** Cache ONLY when a DataFrame is reused across 2+ separate downstream actions.
- **Over-caching Antipattern:** Starves Unified Execution Memory, forcing shuffle spills to disk!

## Streaming foreachBatch Microbatch Processing

Streaming Sink

```python
def process_micro_batch(micro_df, batch_id):
    # Cache microbatch if referenced in multiple merges
    micro_df.persist()

    # ACID Merge microbatch directly into Delta Silver
    (DeltaTable.forName(spark, "silver.txns").alias("t")
     .merge(micro_df.alias("s"), "t.id = s.id")
     .whenMatchedUpdateAll()
     .whenNotMatchedInsertAll()
     .execute())

    micro_df.unpersist()

query = (stream_df.writeStream
    .foreachBatch(process_micro_batch)
    .option("checkpointLocation", "/chk/silver")
    .start())
```

**foreachBatch Superpower:** Allows writing a streaming pipeline into non-streaming sinks, running multi-table merges, and executing deduplications per microbatch.